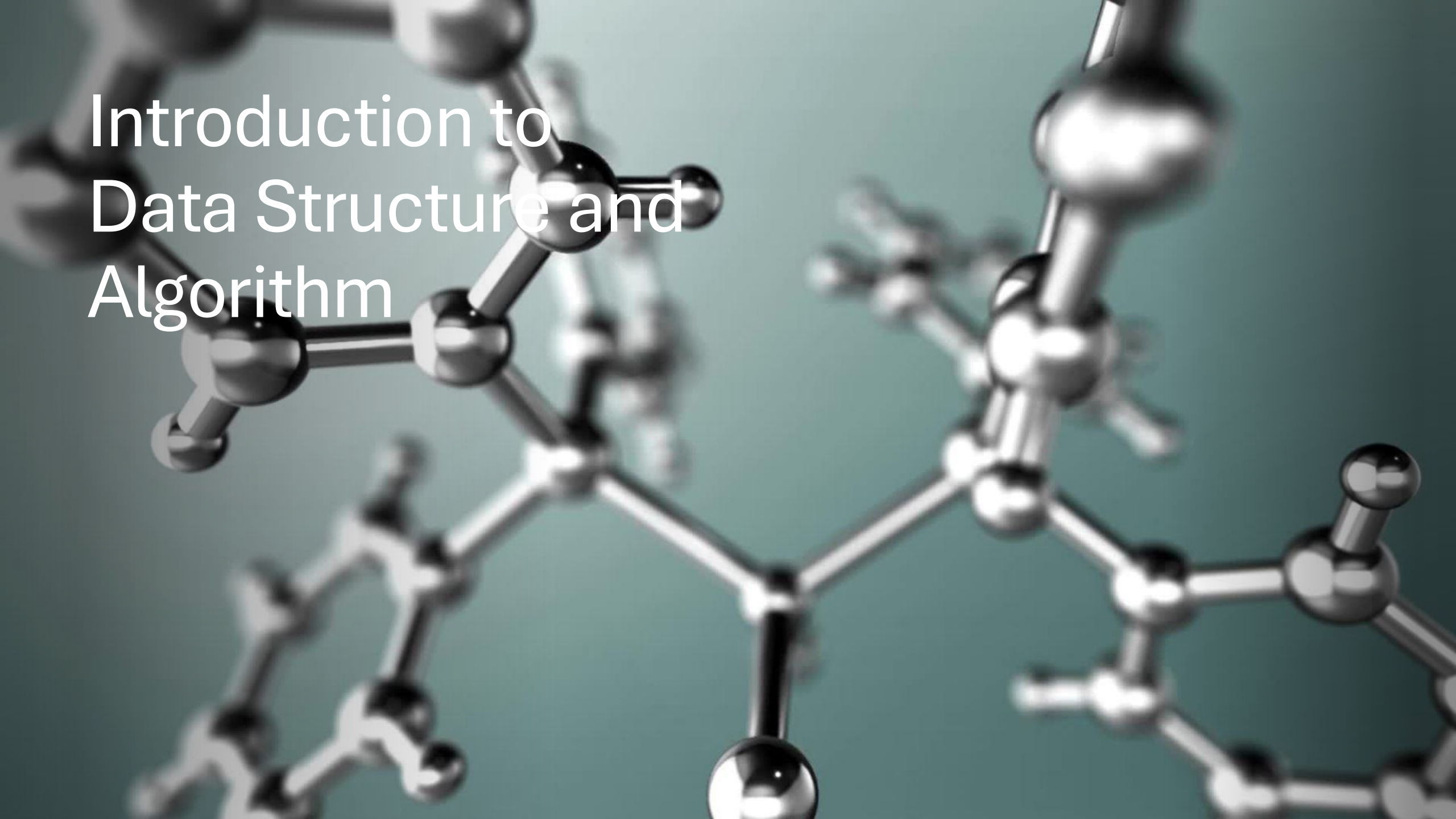


Introduction to Data Structure and Algorithm

Introduction of Data Structure

- Data Structure is a branch of Computer Science. The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program. Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required. The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.
- The scope of a particular data model depends on two factors:

1. First, it must be loaded enough into the structure to reflect the definite correlation of the data with a real-world object.
2. Second, the formation should be so straightforward that one can adapt to process the data efficiently whenever necessary.

Some examples of Data Structures are Arrays, Linked Lists, Stack, Queue, Trees, etc. Data Structures are widely used in almost every aspect of Computer Science, i.e., Compiler Design, Operating Systems, Graphics, Artificial Intelligence, and many more.

Data Structures are the main part of many Computer Science Algorithms as they allow the programmers to manage the data in an effective way. It plays a crucial role in improving the performance of a program or software, as the main objective of the software is to store and retrieve the user's data as fast as possible.

Basic Terminologies related to Data Structures

- Data Structures are the building blocks of any software or program. Selecting the suitable data structure for a program is an extremely challenging task for a programmer.

The following are some fundamental terminologies used whenever the data structures are involved:

- **Data:** We can define data as an elementary value or a collection of values. For example, the Employee's name and ID are the data related to the Employee.
- **Data Items:** A Single unit of value is known as Data Item.

- **Group Items:** Data Items that have subordinate data items are known as Group Items. For example, an employee's name can have a first, middle, and last name.
- **Elementary Items:** Data Items that are unable to divide into sub-items are known as Elementary Items. For example, the ID of an Employee.
- **Entity and Attribute:** A class of certain objects is represented by an Entity. It consists of different Attributes. Each Attribute symbolizes the specific property of that Entity. For example

Attributes	ID	Name	Gender	Job Title
Values	1234	Stacey M. Hill	Female	Software Developer

Understanding the Need for Data Structures

- As applications are becoming more complex and the amount of data is increasing every day, which may lead to problems with data searching, processing speed, multiple requests handling, and many more. Data Structures support different methods to organize, manage, and store data efficiently. With the help of Data Structures, we can easily traverse the data items. Data Structures provide Efficiency, Reusability, and Abstraction.

Why should we learn Data Structures?

- Data Structures and Algorithms are two of the key aspects of Computer Science.
- Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data meaningfully.
- Learning Data Structures and Algorithms will help us become better Programmers.
- We will be able to write code that is more effective and reliable.
- We will also be able to solve problems more quickly and efficiently.

Objectives of Data Structures

Data Structures satisfy two complementary objectives:

- **Correctness:** Data Structures are designed to operate correctly for all kinds of inputs based on the domain of interest. In other words, correctness forms the primary objective of Data Structure, which always depends upon the problems that the Data Structure is meant to solve.
- **Efficiency:** Data Structures also requires to be efficient. It should process the data quickly without utilizing many computer resources like memory space. In a real-time state, the efficiency of a data structure is a key factor in determining the success and failure of the process.

Features of Data Structures

- **Robustness:** Generally, all computer programmers aim to produce software that yields correct output for every possible input, along with efficient execution on all hardware platforms. This type of robust software must manage both valid and invalid inputs.
- **Adaptability:** Building software applications like Web Browsers, Word Processors, and Internet Search Engine include huge software systems that require correct and efficient working or execution for many years. Moreover, software evolves due to emerging technologies or ever-changing market conditions.
- **Reusability:** The features like Reusability and Adaptability go hand in hand. It is known that the programmer needs many resources to build any software, making it a costly enterprise. However, if the software is developed in a reusable and adaptable way, then it can be applied in most future applications. Thus, by executing quality data structures, it is possible to build reusable software, which appears to be cost-effective and timesaving.

ADT of Data Structure

- As per the National Institute of Standards and Technology (NIST), a data structure is an arrangement of information, generally in the memory, for better algorithm efficiency. Data Structures include linked lists, stacks, queues, trees, and dictionaries. They could also be a theoretical entity, like the name and address of a person.

From the definition mentioned above, we can conclude that the operations in data structure include:

- A high level of abstractions like addition or deletion of an item from a list.
- Searching and sorting an item in a list.
- Accessing the highest priority item in a list.

Whenever the data structure does such operations, it is known as an Abstract Data Type (ADT).

- We can define it as a set of data elements along with the operations on the data. The term "abstract" refers to the fact that the data and the fundamental operations defined on it are being studied independently of their implementation. It includes what we can do with the data, not how we can do it.
- An ADI implementation contains a storage structure in order to store the data elements and algorithms for fundamental operation. All the data structures, like an array, linked list, queue, stack, etc., are examples of ADT.

Advantages of using ADTs

- In the real world, programs evolve as a consequence of new constraints or requirements, so modifying a program generally requires a change in one or multiple data structures. For example, suppose we want to insert a new field into an employee's record to keep track of more details about each employee. In that case, we can improve the efficiency of the program by replacing an Array with a Linked structure. In such a situation, rewriting every procedure that utilizes the modified structure is unsuitable. Hence, a better alternative is to separate a data structure from its implementation information. This is the principle behind the usage of Abstract Data Types (ADT).

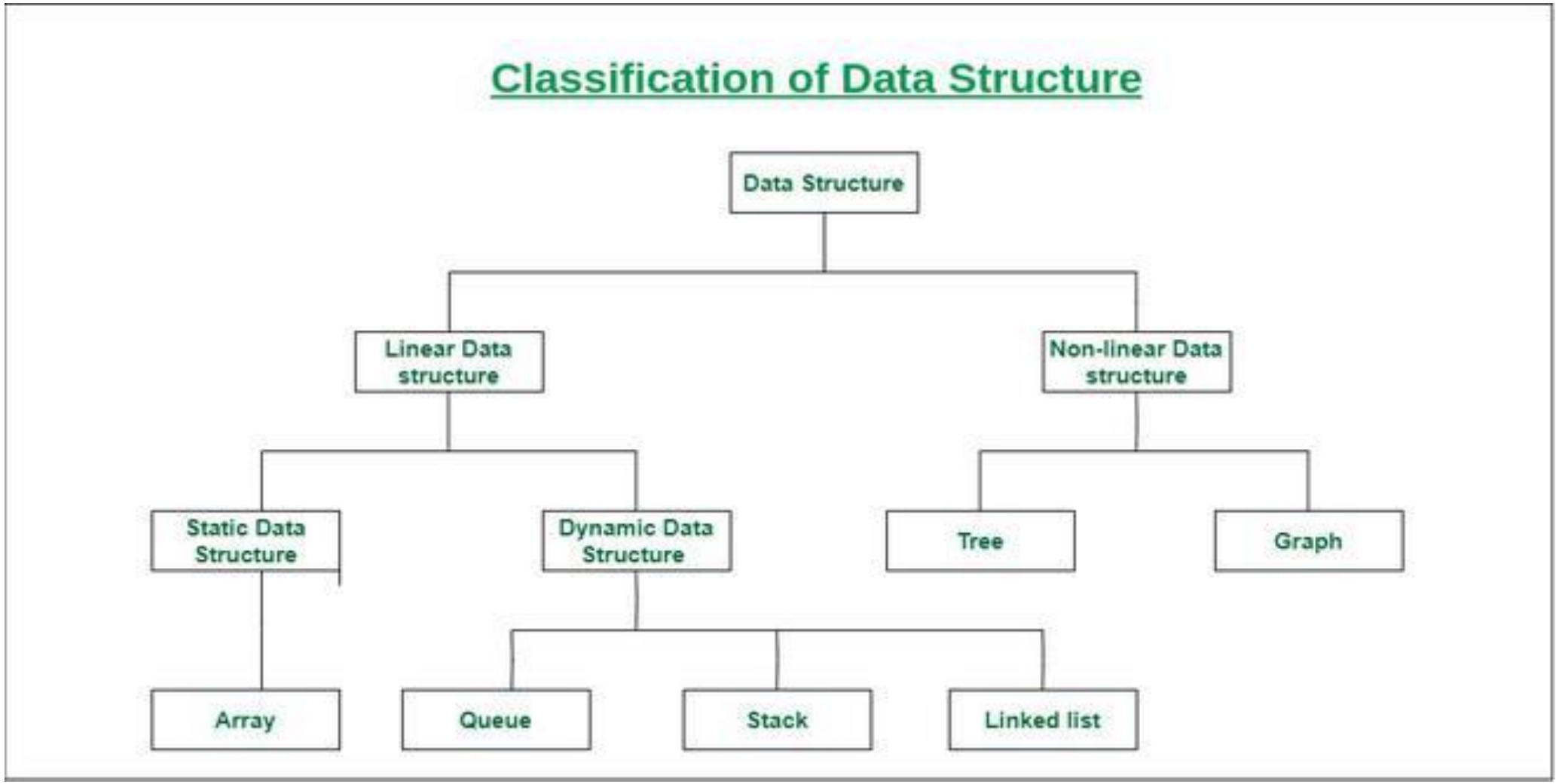
Some Applications of Data Structures

The following are some applications of Data Structures:

- Data Structures help in the organization of data in a computer's memory.
- Data Structures also help in representing the information in databases.
- Data Structures allows the implementation of algorithms to search through data (For example, search engine).
- We can use the Data Structures to implement the algorithms to manipulate data (For example, word processors).
- We can also implement the algorithms to analyse data using Data Structures (For example, data miners).

- Data Structures support algorithms to generate the data (For example, a random number generator).
- Data Structures also support algorithms to compress and decompress the data (For example, a zip utility).
- We can also use Data Structures to implement algorithms to encrypt and decrypt the data (For example, a security system).
- With the help of Data Structures, we can build software that can manage files and directories (For example, a file manager).
- We can also develop software that can render graphics using Data Structures. (For example, a web browser or 3D rendering software).

Classification of Data Structure



Linear data structure

Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure. Examples of linear data structures are array, stack, queue, linked list, etc.

- **Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure. An example of this data structure is an array.
- **Dynamic data structure:** In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code. Examples of this data structure are queue, stack, etc.

Non-linear data structure

- Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can’t traverse all the elements in a single run only.
- Examples of non-linear data structures are trees and graphs.

Atomic

- Stores data at its lowest operational level and is made up of single, non-decomposable data items. Examples of atomic data types include Boolean, string, integer, decimal, and date.
- An **atomic** entity is the smallest unit of data that cannot be broken down further.

Example

- Integer: 5
- Character: 'A'
- Boolean: True

Atomic Variables

- A single integer variable: int age = 25;
- A character variable: char grade = 'A';

Composite

- A composite entity is a combination of multiple atomic or other composite entities grouped together to represent more complex information.

Example :

- Array: An array of integers: int numbers[5] = {1, 2, 3, 4, 5};
- Linked List : Each node contains atomic data (like an integer) and a pointer to the next node.
- Trees: A hierarchical composite structure with nodes containing atomic data and links to child nodes.

Time Complexity

- Time complexity measures the amount of time an algorithm takes to complete as a function of the size of the input n.
- It is represented using Big-O notation (e.g., $O(1)$, $O(n)$, $O(\log n)$).
- Time complexity measures how many operations an algorithm completes in relation to the size of the input. It aids in our analysis of the algorithm's performance scaling with increasing input size. Big O notation (O()) is the notation that is most frequently used to indicate temporal complexity. It offers an upper bound on how quickly an algorithm's execution time will increase.

Best, Worst, and Average Case Complexity:

In analyzing algorithms, we consider three types of time complexity:

- **Best-case complexity (O(best)):** This represents the minimum time required for an algorithm to complete when given the optimal input. It denotes an algorithm operating at its peak efficiency under ideal circumstances.
- Example: Linear Search
- In linear search, you look for a specific value in an array by checking each element one by one.

```
int LinearSearch(int[] arr, int target) {
    if (arr[0] == target) return 0; // Found in 1 step (Best Case)
    ...
}
```

- **Best-case input:** The desired value is the first element in the array.
- **Time Complexity :**o(1), because the algorithm only needs one comparison to find the value.

- **Worst-case complexity (O(worst)):** This denotes the maximum time an algorithm will take to finish for any given input. It represents the scenario where the algorithm encounters the most unfavorable input.
- Example: Linear Search

```
int LinearSearch(int[] arr, int target) {
   for (int i = 0; i < arr.Length; i++) {
      if (arr[i] == target)
          {
             return i;
          } // Check every element (Worst Case)
   }
   return -1; // Target not found
}
```

- **Worst-case input:** The desired value is the last element in the array, or it doesn't exist.

- **Time complexity:** O(n), because the algorithm must check every element in the array.

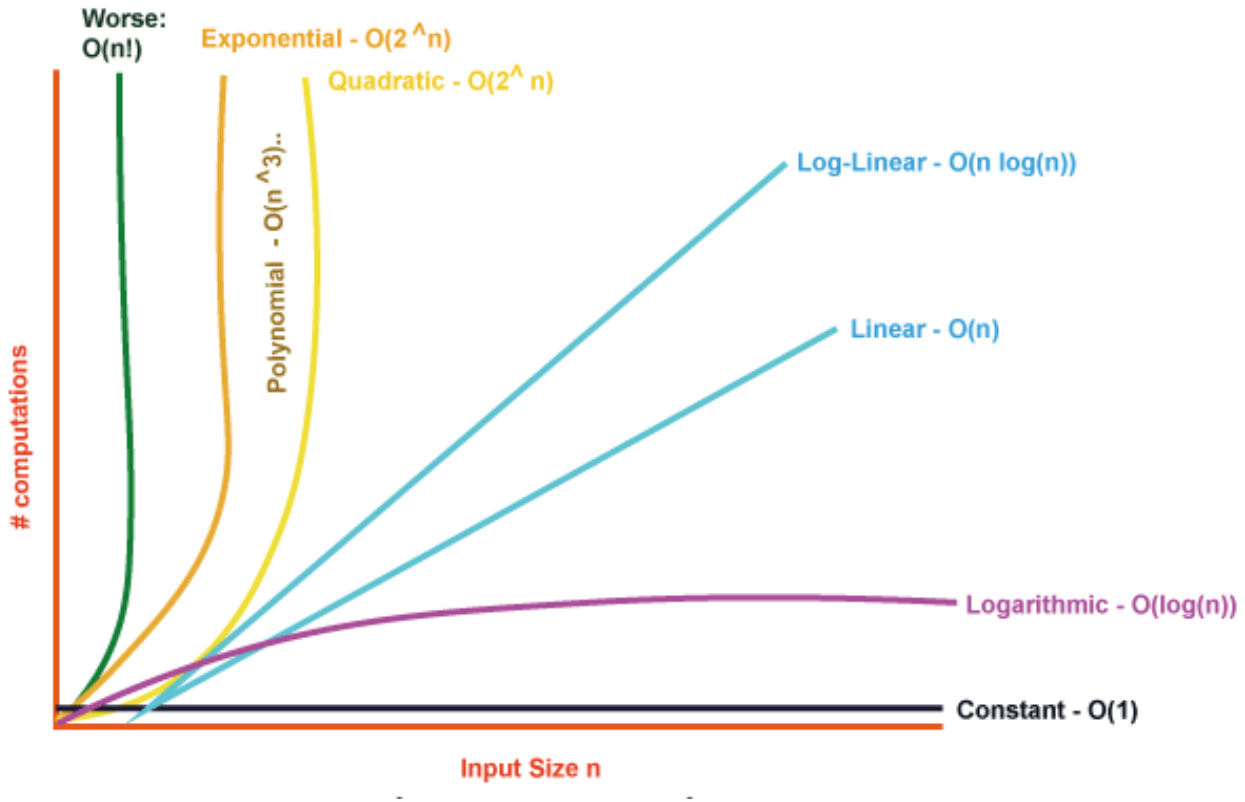
- **Average-case complexity (O(average)):** This estimates the typical running time of an algorithm when averaged over all possible inputs. It provides a more realistic evaluation of an algorithm's performance.
- Example: Linear Search
- If the target value can be at any position in the array with equal probability, the algorithm may need to check, on average, half the elements in the array.

```
int LinearSearch(int[] arr, int target) {
    // On average, the target is found halfway through the array
    ...
}
```

- Example: Linear Search
- If the target value can be at any position in the array with equal probability, the algorithm may need to check, on average, half the elements in the array.
- **Average case:** O(n/2), which simplifies to O(n) in Big-O notation.

Big O Notation:

- Time complexity is frequently expressed using Big O notation. It represents the maximum possible running time for an algorithm given the size of the input. Let's go through some crucial notations:



O(1) - Constant Time Complexity:

- If an algorithm takes the same amount of time to execute no matter how big the input is, it is said to have constant time complexity. This is the best case scenario as it shows how effective the algorithm is. Examples of operations having constant time complexity include accessing a component of an array or executing simple arithmetic calculations.

```
int constantTimeExample(int arr[], int size) {
    return arr[0];
}
```

- As there is only one operation required to return the first element of the array, the get_First_Element function has an O(1) time complexity.

O(log n) - Logarithmic Time Complexity:

- According to logarithmic time complexity, the execution time increases logarithmically as the input size increases. Algorithms with this complexity are often associated with efficient searching or dividing problems in half at each step. A well-known illustration of an algorithm having logarithmic time complexity is the binary search.

```
int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;
```

```
while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target)    return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1; // Not found
}
```

- The binary Search function has a time complexity of O(log n) as it continuously halves the search space until it finds the target element or determines its absence.

O(n) - Linear Time Complexity:

- According to linear time complexity, the running time grows linearly with the size of the input. When navigating data structures or performing actions on each piece, it is frequently noticed. For example, traversing an array or linked list to find a specific element.

```
int linearTimeExample(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}
```

- The print Array function has a time complexity of O(n) as it iterates over each element in the array to print its value.

O(n^2) - Quadratic Time Complexity:

- An algorithm whose runtime quadratically increases with input size. O(n^2) denotes quadratic time complexity, in which an algorithm's execution time scales quadratically with the amount of the input. This type of time complexity is often observed in algorithms that involve nested iterations or comparisons between multiple elements.

```
void printPairs(int arr[], int size)
{ for (int i = 0; i < size; i++) {
    for (int j = 0; j < size; j++) {
     printf("(%d, %d) ", arr[i], arr[j]);
    }
 }
}
```

- he print_Pairs function has a time complexity of O(n^2) as it performs nested iterations over the array, resulting in a quadratic relationship between the execution time and the input size.

O(2^n) - Exponential Time Complexity:

- An algorithm that exhibits an exponential relationship between the execution time and the input size. Exponential time complexity indicates that the algorithm's execution time doubles with each additional element in the input, making it highly inefficient for larger input sizes. This type of time complexity is often observed in algorithms that involve an exhaustive search or generate all possible combinations.

```
int fibonacci(int n) {
    if (n <= 1)
        return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

- The Fibonacci function has a time complexity of O(2^n) as it recursively calculates the Fibonacci sequence, resulting in an exponential increase in the execution time as the input size increases.